

EXPERIMENT 11 – Memory Allocation Methods

Aim: To implement First Fit, Best Fit, and Worst Fit memory allocation methods.

Program A – First Fit

C Program:

```
#include <stdio.h>

int main()
{
    int blockSize[20], processSize[20], allocation[20];
    int nb, np, i, j;

    printf("Enter Number of Blocks: "); scanf("%d",&nb);
    printf("Enter Number of Processes: "); scanf("%d",&np);
    printf("Enter Block Sizes:\n"); for(i=0;i<nb;i++) scanf("%d",&blockSize[i]);
    printf("Enter Process Sizes:\n"); for(i=0;i<np;i++) scanf("%d",&processSize[i]);

    for(i=0;i<np;i++) allocation[i]=-1;

    for(i=0;i<np;i++)
        for(j=0;j<nb;j++)
            if(blockSize[j] >= processSize[i])
            {
                allocation[i]=j;
                blockSize[j]-=processSize[i];
                break;
            }

    printf("\nProcess No\tProcess Size\tBlock No\n");
    for(i=0;i<np;i++)
    {
        printf("%d\t\t%d\t\t",i+1,processSize[i]);
        if(allocation[i]!=-1) printf("%d\n",allocation[i]+1);
        else printf("Not Allocated\n");
    }

    return 0;
}
```

Output (Blocks: 100 500 200 300 600, Processes: 212 417 112 426):

Process No	Process Size	Block No
1	212	2
2	417	5
3	112	2
4	426	Not Allocated

Program B – Best Fit

```
#include <stdio.h>

int main()
{
    int blockSize[20], processSize[20], allocation[20];
    int nb, np, i, j, bestIdx;

    printf("Enter Number of Blocks: "); scanf("%d",&nb);
    printf("Enter Number of Processes: "); scanf("%d",&np);
    printf("Enter Block Sizes:\n"); for(i=0;i<nb;i++) scanf("%d",&blockSize[i]);
    printf("Enter Process Sizes:\n"); for(i=0;i<np;i++) scanf("%d",&processSize[i]);

    for(i=0;i<np;i++) allocation[i]=-1;

    for(i=0;i<np;i++)
    {
        bestIdx=-1;
        for(j=0;j<nb;j++)
            if(blockSize[j] >= processSize[i])
                if(bestIdx==-1 || blockSize[j] < blockSize[bestIdx]) bestIdx=j;

        if(bestIdx!=-1)
        {
            allocation[i]=bestIdx;
            blockSize[bestIdx]-=processSize[i];
        }
    }
}
```

```

    }
}

printf("\nProcess No\tProcess Size\tBlock No\n");
for(i=0;i<np;i++)
{
    printf("%d\t\t%d\t\t",i+1,processSize[i]);
    if(allocation[i]!=-1) printf("%d\n",allocation[i]+1);
    else printf("Not Allocated\n");
}

return 0;
}

```

Output:

Process No	Process Size	Block No
1	212	4
2	417	2
3	112	3
4	426	5

Program C – Worst Fit

```
#include <stdio.h>
```

```

int main()
{
    int blockSize[20], processSize[20], allocation[20];
    int nb, np, i, j, worstIdx;

    printf("Enter Number of Blocks: "); scanf("%d",&nb);
    printf("Enter Number of Processes: "); scanf("%d",&np);
    printf("Enter Block Sizes:\n"); for(i=0;i<nb;i++) scanf("%d",&blockSize[i]);
    printf("Enter Process Sizes:\n"); for(i=0;i<np;i++) scanf("%d",&processSize[i]);

    for(i=0;i<np;i++) allocation[i]=-1;

    for(i=0;i<np;i++)
    {
        worstIdx=-1;
        for(j=0;j<nb;j++)
            if(blockSize[j] >= processSize[i])
                if(worstIdx==-1 || blockSize[j] > blockSize[worstIdx]) worstIdx=j;

        if(worstIdx!=-1)
        {
            allocation[i]=worstIdx;
            blockSize[worstIdx]-=processSize[i];
        }
    }

    printf("\nProcess No\tProcess Size\tBlock No\n");
    for(i=0;i<np;i++)
    {
        printf("%d\t\t%d\t\t",i+1,processSize[i]);
        if(allocation[i]!=-1) printf("%d\n",allocation[i]+1);
        else printf("Not Allocated\n");
    }

    return 0;
}

```

Output:

Process No	Process Size	Block No
1	212	5
2	417	2
3	112	5
4	426	Not Allocated

Result: First Fit, Best Fit, and Worst Fit memory allocation methods were implemented and verified.
